P9 威士忌二級市場觀測系統：專案報告

從非結構化數據分析

張凱傑 2026/03/10

第一章：問題定義與專案動機

1.1 威士忌二級市場現況與資訊痛點

- 痛點一：價格散落於 P9 討論區，缺乏結構化整理。
- 痛點二：手動搜尋與比對成本極高，容易錯失交易機會。
- 痛點三：買賣雙方缺乏客觀的即時行情公定價指標。

圖 1：P9 論壇上非結構化且繁雜的報價資訊截圖

麥卡倫12 關鍵字 威士忌 搜尋 關鍵字：麥卡倫12 討論區：威士忌

我要發表主題 全部主題 品飲心得 單一純麥 其它 討價還價 精華區

排序方式：最後回應時間 1 2 3 4 下頁 > 末頁

最新回應時間	討論主題		發表人	篇數	人氣
2026/03/10 20:06:38	[討價還價] $ 微 麥卡倫12黑盒(M12單桶)	還剩 6 天 17 時 40 分	小十	3	157
2026/03/10 19:26:59	[討價還價] $ 麥卡倫12年 110 proof	還剩 6 天 20 時 22 分	angie37	1	180
2026/03/10 16:59:25	[討價還價] $ 麥卡倫12年 110 proof	還剩 3 天 20 時 3 分	angie37	10	1237
2026/03/10 15:59:41	[討價還價] $ 微 麥卡倫12單桶	還剩 1 天 20 時 47 分	飛翠	4	989
2026/03/10 13:30:15	[討價還價] $ 麥卡倫M10華寫版58.2%、麥卡倫12 4500ML、麥卡倫182..	還剩 1 天 20 時 51 分	酒池肉淋	6	1062
2026/03/10 10:48:16	[討價還價] [圖] $ 麥卡倫12年三桶1.75L+麥卡倫15年三桶	還剩 2 天 15 時 33 分	Lovy	4	791
2026/03/09 20:45:47	[討價還價] [圖] $ 麥卡倫12年原酒110PROOF昇司貨	還剩 0 天 14 時 5 分	大三元發發發	11	2417
2026/03/09 19:26:24	[討價還價] [圖] $ 麥卡倫12年原酒110PROOF昇司版	還剩 5 天 22 時 21 分	李采妮	0	364
2026/03/09 16:03:30	[討價還價] [圖] $ 麥卡倫12年原酒Proof	還剩 5 天 12 時 31 分	樂事雞汁	1	758
2026/03/09 10:36:17	[討價還價] [圖] $ 跑 麥卡倫12年 舊版	還剩 2 天 17 時 20 分	長毛象	24	1416

圖 2：P9 論壇上非結構化且繁雜的報價資訊截圖

1.2 專案動機與解決方案願景

- 核心目標：「輸入酒名 ➔ 一眼看漲跌」的自動化觀測站。
- 預期效益：降低資訊不對稱，打造即時二級市場資料庫。

第二章：技術架構與工程細節

2.1 系統架構總覽 (Data Pipeline)

- **說明從爬蟲 (Requests/BS4) ➔ 清洗 (Pandas/Regex) ➔ 儲存 (SQLite) ➔ 展示 (Streamlit) 的資料流向。**

表格 1：：全自動化數據管線架構圖

2.2 資料萃取與清洗邏輯

- 展示如何使用 Regex 正規表示式，從混亂標題中精準拆解出

```python
#資料清洗
def parse_product_info(row):
    raw_name_text = str(row.get('product_name',''))

    # 1. 品牌解析
    clean_product_name = ''
    if raw_name_text:
        # 移除尾部括號與亂碼
        temp_text = re.sub(r'[(\[\{]+$', '', raw_name_text)
        # 只保留有意義的字元
        clean_product_name = re.sub(r'[^\u4e00-\u9fa5a-zA-Z0-9\s\.\-\(\)]', ' ', temp_text).strip()

    search_text = clean_product_name

    #品牌解析
    resolved_brand = 'Other'
    if search_text:
        resolved_brand = normalize_brand(search_text)


    #年分解析
    year_str = '無年份'
    resolved_year = None

    if clean_product_name:
        year_search_text = BOTTLE_PATTERN.sub('', search_text)
        year_search_text = re.sub(r'\d+\s*(?:週年|周年|th\s*anniversary|th)', '', year_search_text, flags=re.IGNORECASE)
        NOISE_PATTERN = re.compile(r'(?:第|batch|lot|no\.?|vol|ver)\s*\d+(?:\s*批次|\s*版|\s*batch)?', re.IGNORECASE)
        year_search_text = NOISE_PATTERN.sub('', year_search_text)

        explicit_match = re.search(r'(?<!\d)(\d{1,2})\s*(?:年|yo|y)', year_search_text, re.IGNORECASE)
        if explicit_match:
            try:
                val = int(explicit_match.group(1))
                if 5 <= val <= 50: resolved_year = val
            except: pass
        if resolved_year is None:
            numbers = re.findall(r'(\d+)', year_search_text)
            for num_str in numbers:
                val = int (num_str)
                if val in TARGET_YEARS:
                    resolved_year = val
                    break
    if resolved_year: year_str = str(resolved_year)
```

圖 3 ：清洗品牌名稱及年份的程式碼截圖

```python
    # 抓取桶型及系列
    resolved_series = ''
    resolved_style = ''
    # 解析系列
    series_found = extract_attribute(search_text, PRODUCT_MAPPING, SORTED_PRODUCT_KEYS)
    if series_found:
        resolved_series = series_found
    # 解析桶型
    style_found = extract_attribute(search_text, STYLE_MAPPING, SORTED_STYLE_KEYS)
    if style_found:
        resolved_style = style_found
    if resolved_series and resolved_style:
            if resolved_series == resolved_style:
                resolved_style = ''


    return {
        '品牌': resolved_brand,
        '年份': year_str,
        '系列': resolved_series,
        '桶型': resolved_style,
    }
```

圖４：清洗桶型及系列的程式碼截圖

```python
# 計算單價
def calculate_unit_price(row):
    # 1.取得原始價格
    try:
        raw_price = row.get('price', 0)
        if isinstance(raw_price, str):
            raw_price = re.sub(r'[^\d.]','', raw_price)
        original_price = float(raw_price)
    except:
        return 0

    if original_price == 0 : return 0

    # 2.鎖定 product_name，轉小寫以利比對
    search_text = str(row.get('product_name','')).strip().lower()

    if not search_text:
        return int(original_price)

    explicit_qty_match = re.search(r'[/⁄]\s*(\d+)(?:\s*(?:b|瓶|支|入|罐|組|位))?(?:$|\s|[^\w])', search_text)
    if explicit_qty_match:
        qty = int(explicit_qty_match.group(1))
        if qty > 0:
            return int(original_price / qty)

    resolved_quantity = 1
    # 策略 A: X 偵測 (解決 1公升X2, 0.5x2)
    # 規則：X 前面允許是 數字/中文/單位，但不可以是普通英文單字
    x_match = re.search(r'(?:^|[\s\(\)/,.\-\d\u4e00-\u9fa5lm])([x\*])\s*(\d+)', search_text)

    if x_match:
        num = int(x_match.group(2))
        if 1 < num <= 60:
            resolved_quantity = num

```

圖５：清洗單價的程式碼截圖（１）

```python
    # 策略 B: 數字加B (解決6B, 一箱6B)

    if resolved_quantity == 1:
        b_match = re.search(r'(\d+)\s*b(?![a-z])', search_text)
        if b_match:
            num = int(b_match.group(1))
            if 1 < num <= 60:
                resolved_quantity = num

    # 策略 C: 關鍵字 "跑" (社團術語)
    if resolved_quantity == 1:
        pao_match = re.search(r'跑\s*(\d+)', search_text)
        if pao_match:
            num = int(pao_match.group(1))
            if num > 1 : resolved_quantity = num

    # 策略 D: 標準單位 (位, 瓶, 入, 支, 罐)
    if resolved_quantity == 1:
        unit_matches = re.findall(r'(\d+)\s*(?:瓶|入|支|罐|組|位)', search_text)
        for val in unit_matches:
            num = int(val)
            if 1 < num <= 60:
                resolved_quantity = num
                break

    # 策略 E: 中文數字 (兩瓶, 對瓶)
    if resolved_quantity == 1:
        if re.search(r'(?:對)(?:瓶|支|入|b|組|位)?', search_text) and '雙桶' not in search_text:
            resolved_quantity = 2
        else:
            cn_map = {'兩':2, '二':2, '三':3, '四':4, '五':5, '六':6, '十二':12}
            cn_match = re.search(r'([兩二三四五六])\s*(?:瓶|b|支|入|組|位)', search_text)
            if cn_match:
                val = cn_match.group(1)
                resolved_quantity = cn_map.get(val, 1)

```

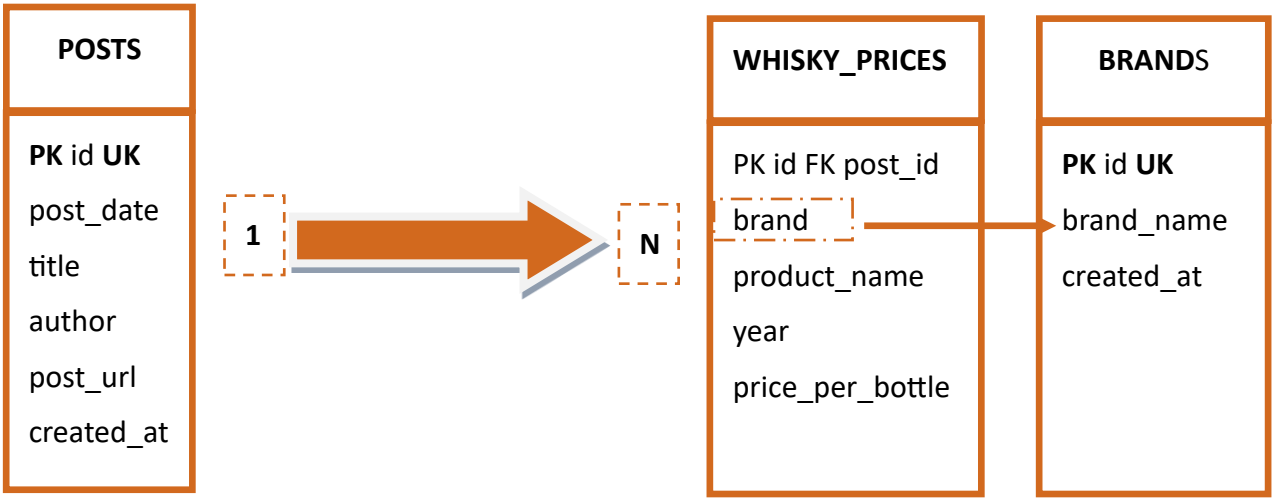
圖 6 ：清洗單價的程式碼截圖（ 2 ）

```python
    # 3. 計算最終單價
    if resolved_quantity > 1:
        calculated_price = int(original_price / resolved_quantity)
        if calculated_price < 400 and original_price > 5000:
            return int(original_price)
        return calculated_price

    return int(original_price)

```

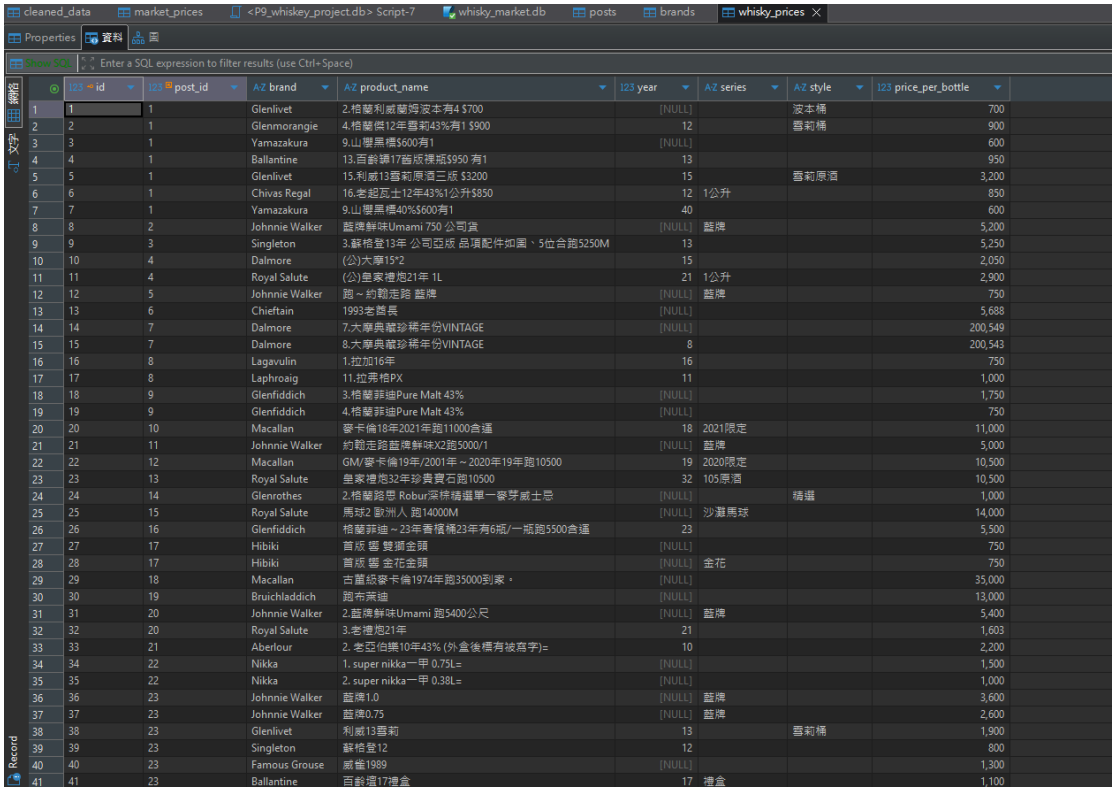
圖 7 ：清洗單價的程式碼截圖（ 3 ）

2.3 關聯式資料庫設計與防重複機制

- 說明 SQLite 的 Table 關聯設計。
- 技術克服：詳細說明如何透過 SQL 語法阻擋爬蟲重複寫入相同報價。
-



- 表格 2 ：：全自動化數據管線架構圖



id	post_id	brand	product_name	year	series	style	price_per_bottle
1	1	Glenlivet	2.格蘭利威蘭姆波本有4 $700	[NULL]		波本桶	700
2	1	Glenmorangie	4.格蘭傑12年雪莉43%有1 $900	12		雪莉桶	900
3	1	Yamazakura	9.山櫻黑標$600有1	[NULL]			600
4	1	Ballantine	13.百齡鑽17酒版裸瓶$950 有1	13			950
5	1	Glenlivet	15.利威13雪莉原酒三版 $3200	15		雪莉原酒	3,200
6	1	Chivas Regal	16.老超瓦士12年43%1公升$850	12	1公升		850
7	1	Yamazakura	9.山櫻黑標40%$600有1	40			600
8	2	Johnnie Walker	藍牌鮮味Umami 750 公司貨	[NULL]	藍牌		5,200
9	3	Singleton	3.蘇格登13年 公司亞版 品項配件如圖、5位台跑5250M	13			5,250
10	4	Dalmore	(公)大摩15*2	15			2,050
11	4	Royal Salute	(公)皇家禮炮21年 1L	21	1公升		2,900
12	5	Johnnie Walker	跑～約翰走路 藍牌	[NULL]	藍牌		750
13	6	Chieftain	1993老酋長	[NULL]			5,688
14	7	Dalmore	7.大摩典藏珍稀年份VINTAGE	[NULL]			200,549
15	7	Dalmore	8.大摩典藏珍稀年份VINTAGE	8			200,543
16	8	Lagavulin	1.拉加16年	16			750
17	8	Laphroaig	11.拉弗格PX	11			1,000
18	9	Glenfiddich	3.格蘭菲迪Pure Malt 43%	[NULL]			1,750
19	9	Glenfiddich	4.格蘭菲迪Pure Malt 43%	[NULL]			750
20	10	Macallan	麥卡倫18年2021年跑11000含運	18	2021限定		11,000
21	11	Johnnie Walker	約翰走路藍牌鮮味X2跑5000/1	[NULL]	藍牌		5,000
22	12	Macallan	GM/麥卡倫19年/2001年～2020年19年跑10500	19	2020限定		10,500
23	13	Royal Salute	皇家禮炮32年珍貴寶石跑10500	32	105原酒		10,500
24	14	Glenrothes	2.格蘭路思 Robur深棕精選單一麥芽威士忌	[NULL]		精選	1,000
25	15	Royal Salute	馬球2 歐洲人 跑14000M	[NULL]	沙灘馬球		14,000
26	16	Glenfiddich	格蘭菲迪～23年香檳桶23年有6瓶/一瓶跑5500含運	23			5,500
27	17	Hibiki	首版 響 雙獅金頭	[NULL]			750
28	17	Hibiki	首版 響 金花金頭	[NULL]	金花		750
29	18	Macallan	古董級麥卡倫1974年跑35000到家。	[NULL]			35,000
30	19	Bruichladdich	跑布萊迪	[NULL]			13,000
31	20	Johnnie Walker	2.藍牌鮮味Umami 跑5400公尺	[NULL]	藍牌		5,400
32	20	Royal Salute	3.老禮炮21年	21			1,603
33	21	Aberlour	2. 老亞伯樂10年43% (外盒後標有被寫字)=	10			2,200
34	22	Nikka	1. super nikka一甲 0.75L=	[NULL]			1,500
35	22	Nikka	2. super nikka一甲 0.38L=	[NULL]			1,000
36	23	Johnnie Walker	藍牌1.0	[NULL]	藍牌		3,600
37	23	Johnnie Walker	藍牌0.75	[NULL]	藍牌		2,600
38	23	Glenlivet	利威13雪莉	13		雪莉桶	1,900
39	23	Singleton	蘇格登12	12			800
40	23	Famous Grouse	威雀1989	[NULL]			1,300
41	23	Ballantine	百齡壇17禮盒	17	禮盒		1,100

圖 8：SQLite 資料庫後端視圖，展示結構化後的報價紀錄與關聯 ID

```
132            if cursor.fetchone():
133                # 資料庫裡已經有這筆資料了，跳過不處理
134                prices_skipped += 1
135            else:
136                # 找不到重複資料，才真正執行寫入
137                cursor.execute('''
138                    INSERT INTO whisky_prices (post_id, brand, product_name, year, series, style, price_per_bottle)
139                    VALUES (?, ?, ?, ?, ?, ?, ?)
140                ''', (post_id, brand_name, product_name, year_db, series, style, price))
141                prices_added += 1
```

圖 9：系統執行日誌 (Terminal Logs)，即時顯示資料庫防重複寫入邏輯之運算過程

第三章：資料成果與規模統計

3.1 資料庫規模統計與 KPI

本系統自建設自動化爬蟲以來，展現穩定的資料蒐集能力。截至 2026 年 3 月，系統已經成功收錄並分析高達 588 篇有效交易貼文。經過正規表達式的深度清洗與交叉過濾，系統已經成功從非結構化的文本中，分析出 482 比具備完整維度(包含品牌、年分、價格與賣家)的有效報價紀錄。

在品牌多樣性方面，本資料庫目前已涵蓋高達 73 個不同的威士忌品牌。

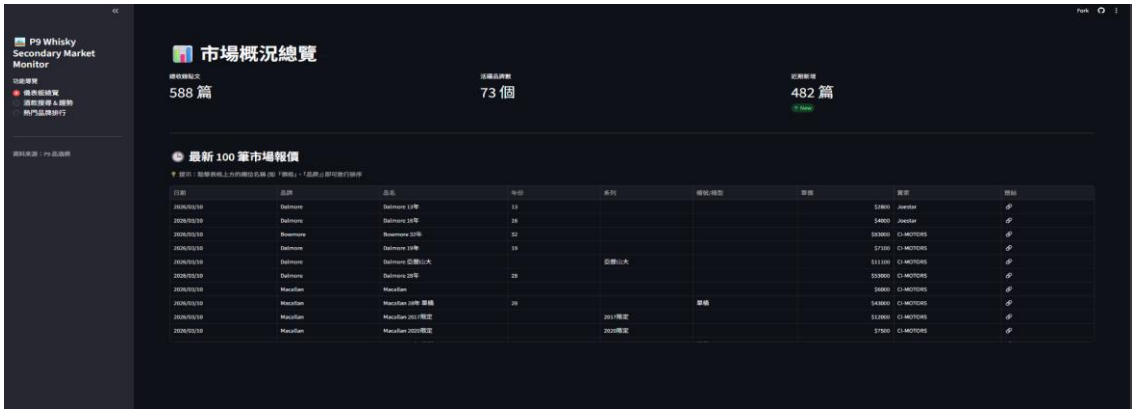


圖 10：系統首頁的 KPI 數據卡片截圖

3.2 市場熱門品牌聲量分析

透過系統後台的彙總運算，我們得以窺見二級市場的流動性與熱度分佈。從「熱門品牌風雲榜」的長條圖中可以明顯觀察到，市場呈現高度集中的頭部效應。其中，麥卡倫 以壓倒性的貼文數與討論聲量穩居市場龍頭，其次為約翰走

路 與大摩等知名主流品牌。這項聲量佔比數據對於市場參與者具有極高的參考價值。高聲量代表著高市場流通性與變現能力，這意味著當藏家或投資客在選擇標的時，若將資金配置於 Top 10 的熱門品牌上，將能大幅降低未來轉手時的流動性風險。

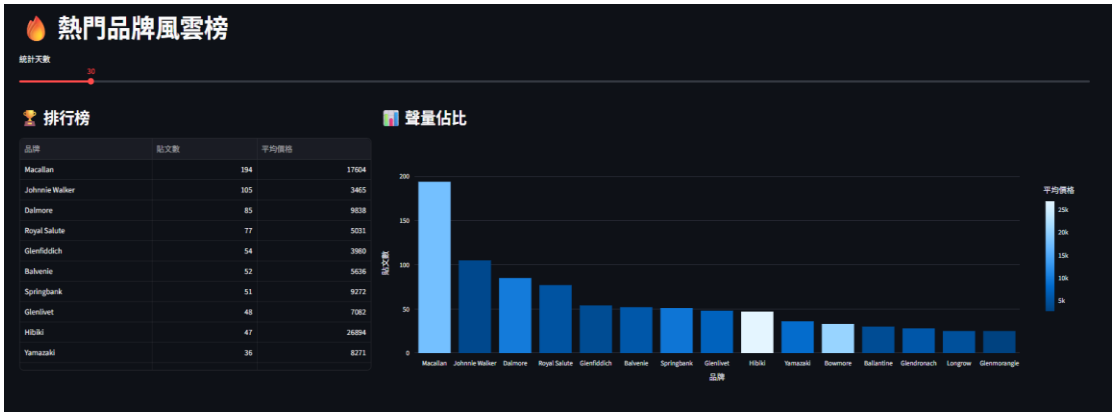


圖 11：門品牌 Top 10 長條圖

3.3 價格趨勢分析案例展示：麥卡倫 12 年

為了驗證系統的行情觀測能力，本專案以市場交易最頻繁的「麥卡倫 12 年」作為分析標的。透過系統產出的價格趨勢散佈圖，我們能清晰地捕捉到該酒款在近兩個月內的真實價格區間。相較於過去買家只能憑藉印象或單一貼文來猜測價格，動態散佈圖成功過濾了極端的高低價，收斂出一條平穩的平均價格基準線。這條基準線徹底破除了買賣雙方在議價時的「報價迷霧」，使得合理的成交區間無所遁形。

圖 12：麥卡倫 12 年價格趨勢散佈圖

圖 13：赫佐本 1 3 年價格趨勢散佈圖

第四章：核心功能與互動操作展示

4.1 儀表板總覽介面

為降低使用者的操作門檻，本系統採用 Streamlit 打造直覺的 Web 互動儀表板。首頁的「市場概況總覽」採用了由上而下的資訊層級設計。最上方安置了核心的 KPI 數據卡片（包含總貼文數、涵蓋品牌數與最新報價筆數），讓使用者在登入的三秒內即可掌握資料庫的

最新運作狀態。下方則配置了「最新 100 筆市場報價」的互動式資料表，提供使用者即時的市場脈動快覽。

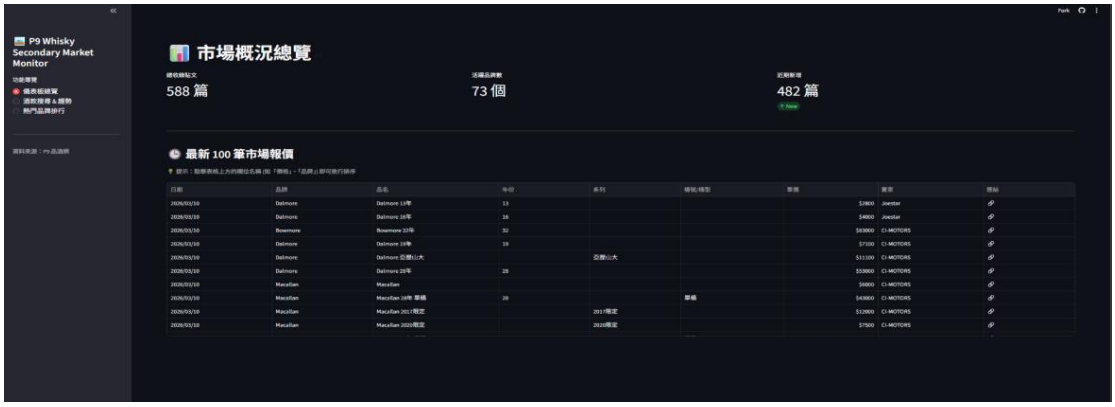


圖 14：Streamlit 系統首頁全螢幕截圖

4.2 多維度進階搜尋引擎

考量到威士忌產品線的複雜度，單一關鍵字搜尋往往會帶出大量無效資訊。本系統開發了「四維交叉搜尋」功能，允許使用者同時輸入「品名（如：響）」、「年份/系列（如：12）」與「桶型」等多重條件進行嚴格篩選。此機制不僅大幅提升了搜尋的精準度，更徹底解決了過去在 P9 論壇中「看得到酒，卻找不到特定年份報價」的嚴重痛點。

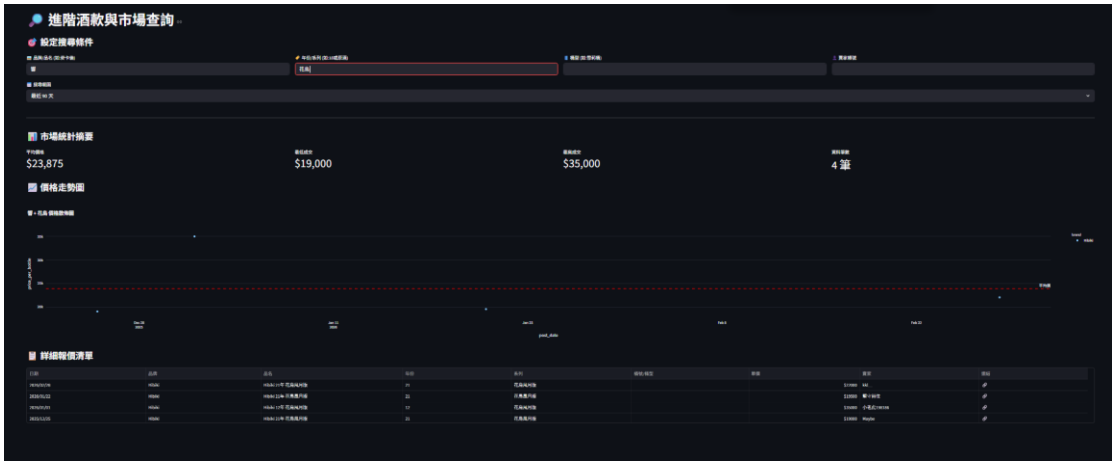


圖 15：進階搜尋條件設定截圖

4.3 動態圖表互動與驗證機制

資料的可信度是金融與交易觀測系統的命脈。在價格走勢圖的設計上，本專案導入了 Plotly 動態視覺化套件。使用者只需將滑鼠懸停 (Hover) 於圖表上的任一資料點，系統便會立即彈出黑色的提示框，顯示該筆報價的賣家帳號、精確日期與詳細金額。

此外，為了確保數據的透明度與可追溯性，每一筆報價清單的右側皆設有「原始連結」按鈕。使用者點擊後，即可直接跳轉回 P9 論壇的原始交易貼文。這項反向查核機制證明了本系統的所有數據皆源自真實市場，絕無造假，賦予了本專案極高的商業公信力。

圖 16：圖表懸停互動特寫截圖

圖 17：詳細報價清單截圖

圖 18：連回 P9 論壇之對照截圖

第五章：商業價值與應用場景

5.1 商業使用者之應用情境

本系統成功將雜亂的網路資訊轉化為具備高商業價值的情報，其應用場景可廣泛覆蓋威士忌生態圈的三大核心客群：

- **收藏家：** 透過直覺的歷史趨勢圖，收藏家能精準判斷進場時機，避免在市場炒作的高點盲目追高，並能在出現低於均價的離群值時，快速出手「撿漏」。

- **酒商：** 系統提供的動態平均價格，具備高度的市場代表性與公信力。這能作為商家在評估收購價，或是設定零售起標價時的重要客觀參考依據。

- **投資客：** 透過熱門品牌風雲榜與聲量佔比統計，投資客能輕易篩選出市場討論度最高、最具增值潛力且具備高流通性的熱門酒款，進行長期的資產配置追蹤。

5.2 深度商業洞察實例剖析：雲頂 10 年

系統的價值不僅在於「呈現數據」，更在於「發現趨勢」。在專案的實機測試階段，透過進階搜尋功能追蹤「雲頂 10 年」的歷史報價時，系統成功捕捉到一項隱蔽的市場異動：同一位賣家在短短一個月內，將該酒款的報價調漲了 10%。這種隱藏在龐大貼文堆中的微小漲幅，若依賴人工肉眼比對幾乎不可能被察覺。本系統成功證明了其具備發掘微觀市場趨勢的強大洞察能力。

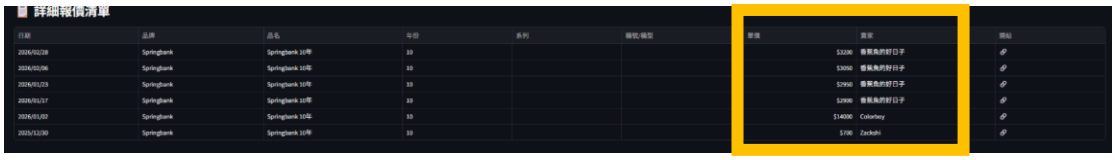


圖 19：捕捉到價格調漲趨勢的進階搜尋與圖表特寫截圖

5.3 本專案之核心技術貢獻

本專案最具指標性的技術成就，在於成功實現了「非結構化文本」向「量化金融數據」的跨越。透過 Regex 清洗演算法與關聯式資料庫設計，系統將混亂不

堪、缺乏標準命名的賣家自由輸入文字，強制統一歸納為標準化的品牌與年份維度。這不僅大幅降低了市場資訊不對稱所造成的交易摩擦成本。

第六章：技術反思與未來規劃

6.1 系統目前限制與挑戰

身為一項處於第一版 (v1.0) 階段的資料工程專案，本系統仍存在部分具備優化空間的限制：

1. **資料來源單一性：** 目前系統的爬蟲範圍僅鎖定 P9 論壇。雖然 P9 為國內最大平台，但仍無法涵蓋 Facebook 私密交流社團或實體拍賣會的交易數據，可能導致部分罕見酒款的樣本數偏低。

2. **極端字串解析瓶頸：** 受限於少數賣家嚴重的拼字錯誤或極度精簡的非標準縮寫，目前 NLP 解析模組對於品牌與年份的捕捉準確率約落在 92% 左右，仍有微小比例的漏網之魚。

3. **開價與成交價的落差：** 系統目前抓取的數據為賣家的「初始開價，尚無法追蹤買賣雙方私下私訊議價後的「最終成交價」，這使得平均價格可能存在微幅的高估溢價。

6.2 未來功能擴充藍圖 (Roadmap)

本專案目前的 v1.0 版本已成功建構了穩定的數據基礎設施，未來將規劃朝以下四個維度進行深度技術擴充，以進一步提升系統的商業價值與使用者體驗 ：

- **多來源數據整合**： 目前系統資料來源侷限於單一論壇，未來計畫擴展爬蟲程式的抓取範圍，將範疇延伸至各大知名拍賣網站、Facebook 私密威士忌交易社團，以及其他二級交流平台 。透過多來源數據的匯整與權重比對，系統將能排除單一平台可能存在的價格偏誤，提供更具市場代表性的「全網綜合參考價」 。
- **深度的酒款數據分析**： 為了滿足進階收藏家的需求，未來將針對單一酒款提供更高維度的數據分析功能 。例如：針對特定單一桶號或特殊版本酒款進行獨立的追蹤，並導入賣家信用評等分析、交易週期預測等進階指標，讓使用者對特定品項的市場流動性有更全面的認知 。
- **AI 價格預測模型**： 系統計畫在累積足夠長期的歷史數據後，導入機器學習模型進行價格與銷量的趨勢預測 。藉由時間序列模型（如 ARIMA 或 Prophet）分析市場週期與季節性波動，系統將能為特定熱門酒款提供未來一季的價格走勢預判，

輔助使用者做出更精準的投資決策 。

- **主動式即時推播提醒**： 為了解決使用者必須主動登入儀表板查閱的限制，未來將開發專屬的手機 App 或串接 LINE Bot 機器人服務 。使用者可於系統中設定目標關注價或「離群低價」自動警示，一旦爬蟲偵測到市場出現符合條件的最新報價貼文，系統將會立即發送主動推播提醒，實現真正的即時市場監控體驗 。

附錄：中英文雙語關鍵詞彙表 (Glossary)

中文專業詞彙（英文專業詞彙）: 於本專案中之應用情境說明

- 二級市場（Secondary Market ）: 泛指收藏家與買賣雙方進行威士忌轉手交易的非官方市場。
- 網路爬蟲（Web Scraping）: 運用 Python (Requests + BeautifulSoup) 自動化獲取 P9 論壇網頁原始碼的技術。
- 資料管線（Data Pipeline）: 涵蓋資料抓取、清洗、關聯儲存至前端展示的全自動化資料處理流程。
- 非結構化資料（Unstructured Dat）: 指論壇中格式不統一、缺乏固定欄位

定義的自由文本報價資訊。

- 正規表示式（Regular Expression (Regex)）：用於資料清洗階段，從混亂字串中精準匹配並萃取出酒款年份、容量與價格的字串比對技術。
- 關聯式資料庫（Relational Database）：本專案採用 SQLite，透過正規化設計將品牌與報價資料分表儲存，確保資料一致性。
- 探索性資料分析（Exploratory Data Analysis (EDA)）：透過彙整總報價筆數、計算平均價格與產出排行榜，初步挖掘市場輪廓的過程。
- 動態儀表板（Interactive Dashboard）：使用 Streamlit 打造的視覺化介面，允許使用者輸入條件並即時改變圖表呈現結果。